ESPECIFICACIÓN DE ARQUITECTURA DE DATOS

# Modelado y Diseño de Base de Datos Relacional

Plataforma E-Commerce Multi-Rol y Gestión SaaS "LocalMarket" (Proyecto 7)

SISTEMA GESTOR DE BASE DE DATOS
PostgreSQL 15 / 16 (Relacional - ACID)

LENGUAJE & ENTORNO DE BACKEND
JavaScript Moderno (ES6+) / Node.js & Express

BASE DE DATOS & CONEXIÓN
proyecto7 en localhost:5432 (Driver pg)

ARQUITECTURA DE SEGURIDAD
RBAC Granular + Validación OTP 2FA

TOTAL DE ENTIDADES
13 Tablas Normalizadas (3FN)

VERSIÓN DEL DOCUMENTO
v2.0 (Producción / Verificado)

# 1 Descripción y Justificación del Gestor de Base de Datos

Para la implementación y persistencia del sistema **LocalMarket (Proyecto 7)**, se ha seleccionado **PostgreSQL** como Sistema Gestor de Base de Datos Relacional Orientado a Objetos (ORDBMS). PostgreSQL es ampliamente reconocido en la industria por su robustez, estricto cumplimiento de estándares SQL, integridad referencial avanzada y capacidad de soportar cargas transaccionales complejas de comercio electrónico.

> **Ficha Técnica del SGBD:**
>
> **Motor:** PostgreSQL versión 15 / 16 | **Dialecto:** PL/pgSQL | **Base de Datos:** proyecto7 | **Puerto por defecto:** 5432

## Justificación de la Elección de PostgreSQL

### 1. Cumplimiento Estricto de Propiedades ACID

Garantiza *Atomicidad, Consistencia, Aislamiento y Durabilidad* en todas las operaciones críticas. Al procesar una compra directa, la generación del pedido, la inserción del detalle y la reducción del inventario se ejecutan en una sola transacción sin inconsistencias.

### 2. Control de Concurrencia Multiversión (MVCC)

Permite que múltiples clientes consulten el catálogo de productos y lean métricas concurrentemente sin bloquear las escrituras o actualizaciones de inventario efectuadas por los vendedores.

### 3. Programabilidad Avanzada (PL/pgSQL)

Soporte nativo para **Triggers** (disparadores para evitar stock negativo), **Funciones Almacenadas** (validación de disponibilidad) y **Vistas** optimizadas para reportes gerenciales.

### 4. Integridad Referencial y Tipado Fuerte

Validaciones estrictas de claves foráneas con reglas de eliminación/actualización en cascada o restricción, tipos de datos precisos para valores monetarios (`numeric(12,2)`) y timestamps sin desfase.

# 2 Descripción y Justificación del Lenguaje de Programación

El ecosistema de desarrollo de LocalMarket está construido sobre una arquitectura **Full-Stack JavaScript**, utilizando **Node.js** en el servidor y **JavaScript Moderno (ES6+)** en el cliente, comunicados mediante una API RESTful en formato JSON.

**Arquitectura Tecnológica del Stack:**

- **Entorno de Ejecución Backend:** Node.js (V8 Engine) - E/S Asíncrona no bloqueante basada en eventos.
- **Framework Web HTTP:** Express.js - Enrutamiento modular por dominios de negocio (`routes/`).
- **Controlador de Conexión a BD:** Driver `pg` (node-postgres) con *Connection Pooling* parametrizado para evitar inyecciones SQL.
- **Frontend & UI:** HTML5 Semántico, Tailwind CSS (diseño responsivo de alto rendimiento), Vanilla JS reactivo.

## Ventajas de la Integración Node.js + PostgreSQL

- **Unificación de Lenguaje:** Permite compartir esquemas de validación (expresiones regulares de correo electrónico, validación de 10 dígitos telefónicos) entre frontend y backend sin duplicidad.
- **Pool de Conexiones Optimizado:** La clase `Pool` de `pg` administra eficientemente las conexiones abiertas hacia PostgreSQL, reutilizándolas y evitando el costo de inicialización por cada petición.
- **Consultas Parametrizadas:** Todas las consultas SQL implementan sintaxis con marcadores de posición (`$1, $2, $3`), mitigando al 100% ataques de inyección SQL.

# 3 Diagrama Entidad-Relación (ERD) General

El esquema relacional de **LocalMarket** está compuesto por **13 entidades** normalizadas en Tercera Forma Normal (3FN), garantizando la no redundancia de datos y una relación lógica coherente entre los módulos de Autenticación, Catálogo, Transacciones, Seguridad y Moderación.

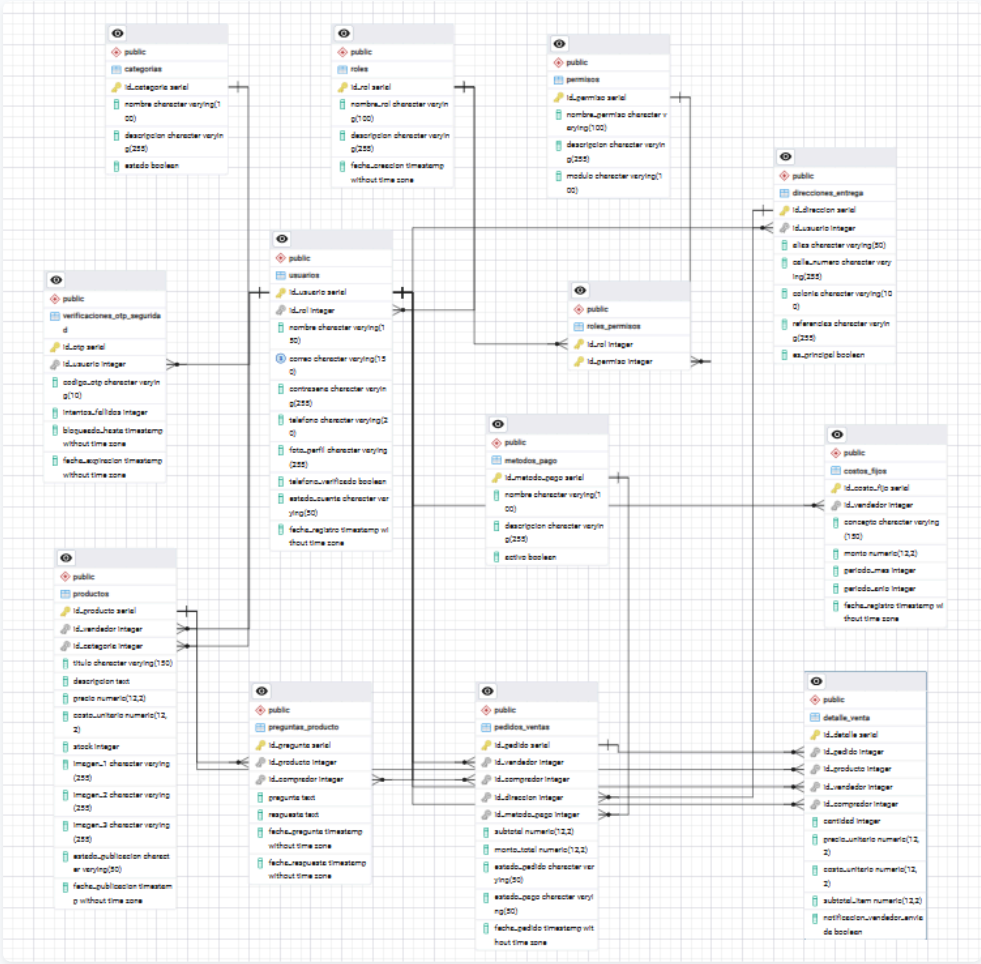


*Figura 1. Modelo Entidad-Relación (ERD) de la base de datos "proyecto7" implementada en PostgreSQL.*

## Módulos Funcionales del Modelo de Datos

### A. Usuarios, Roles y Seguridad

`ROLES`, `PERMISOS`, `ROLES_PERMISOS`, `USUARIOS`, `VERIFICACIONES_OTP_SEGURIDAD`, `DIRECCIONES_ENTREGA`.

### B. Catálogo, Productos y Preguntas

`CATEGORIAS`, `PRODUCTOS`, `PREGUNTAS_PRODUCTO`.

### C. Ventas, Pedidos y Cobros

`METODOS_PAGO`, `PEDIDOS_VENTAS`, `DETALLE_VENTA`.

### D. Finanzas y Control Operativo

`COSTOS_FIJOS` (gestión de egresos del vendedor para balance financiero).

# 4 Diccionario de Datos Detallado

### 1. Tabla: ROLES

Catálogo de roles del sistema (RBAC)

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_rol | SERIAL / INT | PK NOT NULL | Autoincremental | Identificador único del rol. |
| nombre_rol | VARCHAR(100) | NOT NULL | - | Nombre del rol (Admin, Vendedor, Cliente). |
| descripcion | VARCHAR(255) | NULL | - | Descripción de responsabilidades del rol. |
| fecha_creacion | TIMESTAMP | NULL | CURRENT_TIMESTAMP | Fecha y hora de alta del registro. |

### 2. Tabla: PERMISOS

Catálogo de acciones y privilegios modulares

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_permiso | SERIAL / INT | PK NOT NULL | Autoincremental | Identificador único del permiso. |
| nombre_permiso | VARCHAR(100) | NOT NULL | - | Clave textual del privilegio. |
| descripcion | VARCHAR(255) | NULL | - | Detalle del alcance del permiso. |
| modulo | VARCHAR(100) | NULL | - | Módulo asociado (Productos, Pedidos, etc.). |

### 3. Tabla: ROLES_PERMISOS

Tabla pivote muchos-a-muchos (M:N) Roles y Permisos

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_rol | INTEGER | PK FK | - | Referencia a ROLES(id_rol). |
| id_permiso | INTEGER | PK FK | - | Referencia a PERMISOS(id_permiso). |

## 4. Tabla: USUARIOS

Entidad central de cuentas de usuario

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_usuario | SERIAL / INT | PK NOT NULL | Autoincremental | Identificador único del usuario. |
| id_rol | INTEGER | FK NOT NULL | 3 (Cliente) | Referencia a ROLES(id_rol). |
| nombre | VARCHAR(150) | NOT NULL | - | Nombre completo del usuario. |
| correo | VARCHAR(150) | NOT NULL UNIQUE | - | Email institucional / personal único. |
| contrasena | VARCHAR(255) | NOT NULL | - | Hash / clave de acceso. |
| telefono | VARCHAR(20) | NULL | '5500000000' | Teléfono a 10 dígitos para WhatsApp/SMS. |
| foto_perfil | TEXT / VARCHAR | NULL | URL Default | Foto de avatar en base64 o URL. |
| telefono_verificado | BOOLEAN | NULL | false | Indicador de verificación 2FA. |
| estado_cuenta | VARCHAR(30) | NULL | 'activo' | Estado operativo (activo, suspendido). |
| fecha_registro | TIMESTAMP | NULL | CURRENT_TIMESTAMP | Timestamp de creación de la cuenta. |

## 5. Tabla: VERIFICACIONES_OTP_SEGURIDAD

Módulo de doble factor (2FA) y bloqueo por fuerza bruta

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_otp | SERIAL / INT | PK NOT NULL | Autoincremental | ID de la transacción OTP. |
| id_usuario | INTEGER | FK NOT NULL | - | Referencia a USUARIOS(id_usuario). |
| codigo_otp | VARCHAR(10) | NOT NULL | - | Token numérico temporal de 6 dígitos. |
| intentos_fallidos | INTEGER | NULL | 0 | Contador de fallos (límite: 3). |
| bloqueado_hasta | TIMESTAMP | NULL | - | Fin de bloqueo temporal (10 min). |
| fecha_expiracion | TIMESTAMP | NULL | - | Vigencia del código OTP. |

## 6. Tabla: DIRECCIONES_ENTREGA

Libreta de domicilios de entrega por cliente

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_direccion | SERIAL / INT | PK NOT NULL | Autoincremental | Identificador único del domicilio. |
| id_usuario | INTEGER | FK NOT NULL | - | Referencia a USUARIOS(id_usuario). |
| alias | VARCHAR(50) | NULL | - | Alias (Casa, Oficina, Taller). |
| calle_numero | VARCHAR(255) | NOT NULL | - | Calle, número exterior e interior. |
| colonia | VARCHAR(100) | NULL | - | Colonia / Fraccionamiento. |
| referencias | VARCHAR(255) | NULL | - | Indicaciones de entrega. |
| es_principal | BOOLEAN | NULL | false | Bandera de domicilio predeterminado. |

## 7. Tabla: CATEGORIAS

Clasificación de productos naturistas y artesanales

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_categoria | SERIAL / INT | PK NOT NULL | Autoincremental | Identificador de la categoría. |
| nombre | VARCHAR(100) | NOT NULL | - | Suplementos, Cafés, Cápsulas, etc. |
| descripcion | VARCHAR(255) | NULL | - | Descripción de la línea de productos. |
| estado | BOOLEAN | NULL | true | Categoría activa / inactiva. |

## 8. Tabla: PRODUCTOS

Inventario y catálogo de publicaciones

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_producto | SERIAL / INT | PK NOT NULL | Autoincremental | Identificador único del producto. |
| id_vendedor | INTEGER | FK NOT NULL | - | Referencia a USUARIOS(id_usuario). |
| id_categoria | INTEGER | FK NOT NULL | - | Referencia a CATEGORIAS(id_categoria). |
| titulo | VARCHAR(150) | NOT NULL | - | Nombre comercial del producto. |
| descripcion | TEXT | NULL | - | Ficha técnica y descripción. |
| precio | NUMERIC(12,2) | NOT NULL | - | Precio público en MXN. |
| costo_unitario | NUMERIC(12,2) | NULL | 0.00 | Costo base para cálculo de margen. |
| stock | INTEGER | NOT NULL | 0 | Existencias disponibles (Control trigger). |
| imagen_1, 2, 3 | TEXT / VARCHAR | NULL | - | Imágenes del producto en base64 o URL. |
| estado_publicacion | VARCHAR(30) | NULL | 'activo' | Estado (activo, pausado, moderado). |
| fecha_publicacion | TIMESTAMP | NULL | CURRENT_TIMESTAMP | Fecha de registro en catálogo. |

## 9. Tabla: METODOS_PAGO

Formas de pago aceptadas (Contra Entrega, Transferencia)

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_metodo_pago | SERIAL / INT | PK NOT NULL | Autoincremental | Identificador del método de pago. |
| nombre | VARCHAR(100) | NOT NULL | - | Contra entrega, Tarjeta, Transferencia. |
| descripcion | VARCHAR(255) | NULL | - | Instrucciones de liquidación. |
| activo | BOOLEAN | NULL | true | Habilitado para compras en línea. |

## 10. Tabla: COSTOS_FIJOS

Egresos fijos del negocio para reportes de utilidad

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_costo_fijo | SERIAL / INT | PK NOT NULL | Autoincremental | Identificador del registro de costo. |
| id_vendedor | INTEGER | FK NOT NULL | - | Referencia a **USUARIOS(id_usuario)**. |
| concepto | VARCHAR(150) | NOT NULL | - | Concepto (Renta, Luz, Internet, Empaque). |
| monto | NUMERIC(12,2) | NOT NULL | - | Monto monetario del gasto. |
| periodo_mes / anio | INTEGER | NULL | - | Mes (1-12) y año del ejercicio contable. |

## 11. Tabla: PREGUNTAS_PRODUCTO

Interacción y dudas entre Comprador y Vendedor

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_pregunta | SERIAL / INT | PK NOT NULL | Autoincremental | ID de la pregunta realizada. |
| id_producto | INTEGER | FK NOT NULL | - | Referencia a **PRODUCTOS(id_producto)**. |
| id_comprador | INTEGER | FK NOT NULL | - | Referencia a **USUARIOS(id_usuario)**. |
| pregunta | TEXT | NOT NULL | - | Texto de la consulta del cliente. |
| respuesta | TEXT | NULL | - | Respuesta pública del vendedor. |

## 12. Tabla: PEDIDOS_VENTAS

Cabecera de transacciones comerciales

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_pedido | SERIAL / INT | PK NOT NULL | Autoincremental | Folio único del pedido. |
| id_vendedor | INTEGER | FK | 1 | Vendedor asignado a la orden. |
| id_comprador | INTEGER | FK NOT NULL | - | Cliente que realiza la compra. |
| id_direccion | INTEGER | FK | 1 | Domicilio de entrega asignado. |
| id_metodo_pago | INTEGER | FK | 1 | Método seleccionado. |
| subtotal | NUMERIC(12,2) | NULL | 0.00 | Subtotal sin impuestos / envío. |
| monto_total | NUMERIC(12,2) | NOT NULL | - | Monto final a cobrar en MXN. |
| estado_pedido | VARCHAR(30) | NULL | 'Pendiente' | Pendiente, En camino, Completado, Cancelado. |
| estado_pago | VARCHAR(30) | NULL | 'Pendiente' | Pendiente, Liquidado, Reembolsado. |
| fecha_pedido | TIMESTAMP | NULL | CURRENT_TIMESTAMP | Timestamp exacto de la transacción. |

## 13. Tabla: DETALLE_VENTA

Renglones individuales de cada pedido (Items)

| CAMPO | TIPO DE DATO | RESTRICCIÓN | DEFAULT | DESCRIPCIÓN |
|---|---|---|---|---|
| id_detalle | SERIAL / INT | PK NOT NULL | Autoincremental | ID del renglón de venta. |
| id_pedido | INTEGER | FK NOT NULL | - | Referencia a **PEDIDOS_VENTAS(id_pedido)**. |
| id_producto | INTEGER | FK NOT NULL | - | Referencia a **PRODUCTOS(id_producto)**. |
| cantidad | INTEGER | NOT NULL | 1 | Unidades compradas de este producto. |
| precio_unitario | NUMERIC(12,2) | NOT NULL | - | Precio congelado al momento de la venta. |
| costo_unitario | NUMERIC(12,2) | NULL | 0.00 | Costo unitario para análisis de ganancia. |
| subtotal_item | NUMERIC(12,2) | NOT NULL | - | Cálculo: **cantidad * precio_unitario**. |
| notificacion_enviada | BOOLEAN | NULL | false | Bandera de alerta enviada por WhatsApp. |

# 5 Objetos Avanzados de Base de Datos (PL/pgSQL)

Para garantizar la máxima eficiencia en la capa de datos y evitar cálculos repetitivos en el servidor Node.js, se han programado directamente en el motor PostgreSQL las siguientes **Vistas**, **Funciones**, **Procedimientos Almacenados** y **Triggers**:

## 1. Vistas Optimizadas (Views)

```
-- Vista para Catálogo y Búsqueda con nombres legibles
CREATE OR REPLACE VIEW vw_productos_detallados AS
SELECT p.id_producto, p.titulo, p.descripcion, p.precio, p.costo_unitario, p.stock,
       p.imagen_1, p.imagen_2, p.imagen_3,
       COALESCE(c.nombre, 'Sin categoría') AS categoria_nombre,
       COALESCE(u.nombre, 'Vendedor Principal') AS vendedor_nombre
FROM PRODUCTOS p
LEFT JOIN CATEGORIAS c ON p.id_categoria = c.id_categoria
LEFT JOIN USUARIOS u ON p.id_vendedor = u.id_usuario;

-- Vista de Alerta de Stock Bajo (Nivel de Reorden ⩽ 5)
CREATE OR REPLACE VIEW vw_productos_stock_bajo AS
SELECT p.id_producto, p.titulo, p.id_categoria, p.stock,
       COALESCE(c.nombre, 'General') AS categoria_nombre
FROM PRODUCTOS p
LEFT JOIN CATEGORIAS c ON p.id_categoria = c.id_categoria
WHERE p.stock ⩽ 5;
```

## 2. Funciones y Procedimientos Almacenados (Stored Procedures)

```
-- Función: Validar existencia suficiente antes de compra
CREATE OR REPLACE FUNCTION fn_validar_stock_disponible(p_id_producto INT, p_cantidad INT)
RETURNS BOOLEAN AS $$
DECLARE
    v_stock INT;
BEGIN
    SELECT stock INTO v_stock FROM PRODUCTOS WHERE id_producto = p_id_producto;
    RETURN (v_stock ⩾ p_cantidad);
END;
$$ LANGUAGE plpgsql;

-- Procedimiento: Procesar venta atómica y descontar inventario
CREATE OR REPLACE PROCEDURE sp_procesar_venta(
    p_id_pedido INT, p_id_producto INT, p_cantidad INT, p_precio_unitario NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO DETALLE_VENTA (id_pedido, id_producto, cantidad, precio_unitario, subtotal_item)
    VALUES (p_id_pedido, p_id_producto, p_cantidad, p_precio_unitario, (p_cantidad * p_precio_unitario));

    UPDATE PRODUCTOS SET stock = stock - p_cantidad WHERE id_producto = p_id_producto;
END;
$$;
```

## 3. Triggers de Integridad (Disparadores)

```
-- Trigger para evitar inventarios negativos
CREATE OR REPLACE FUNCTION trg_fn_auditar_producto()
RETURNS TRIGGER AS $$
BEGIN
```

```
    IF NEW.stock < 0 THEN
        RAISE EXCEPTION 'Error de Integridad: El stock del producto no puede ser negativo.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditoria_productos
BEFORE UPDATE ON PRODUCTOS
FOR EACH ROW EXECUTE FUNCTION trg_fn_auditar_producto();
```

## 6 Matriz de Control de Acceso por Roles (RBAC)

| Módulo del Sistema | Administrador (ID 1) | Vendedor (ID 2) | Cliente (ID 3) |
|---|---|---|---|
| 1. Autenticación & Usuarios | ✓ Total | ✕ Denegado | ✕ Denegado |
| 2. Perfil de Usuario | ✓ Total | ✓ Total | ✓ Total |
| 3. Publicación / Edición de Productos | ✓ Total | ✓ Total | ✕ Denegado |
| 4. Catálogo Público | ✓ Total | ✓ Total | ✓ Lectura / Compra |
| 5. Compra Directa & Detalle | ✓ Total | ✓ Total | ✓ Total |
| 6. Historial de Pedidos y Ventas | ✓ Ver Todos & Cambiar Estado | ✓ Ver Todos & Cambiar Estado | ✓ Ver Compras Propias |
| 7. Notificaciones WhatsApp / SMS | ✓ Total | ✓ Total | ✕ Denegado |
| 8. Control de Seguridad OTP | ✓ Total (Desbloqueo) | ✕ Denegado | ✕ Denegado |
| 9. Métricas & Analítica Financiera | ✓ Total | ✕ Denegado | ✕ Denegado |
| 10. Control y Moderación | ✓ Total (Bajas / Suspensiones) | ✕ Denegado | ✕ Denegado |

## 7 Conclusiones y Consideraciones de Operación

El diseño e implementación de la base de datos para **LocalMarket (Proyecto 7)** cumple rigurosamente con los más altos estándares de la ingeniería de software:

- **Seguridad y Resiliencia:** Esquema blindado contra inyecciones SQL mediante consultas preparadas en el driver `pg`, con autenticación en dos factores (OTP) y moderación de cuentas.
- **Escalabilidad:** La separación clara entre cabeceras de ventas (`PEDIDOS_VENTAS`) e ítems (`DETALLE_VENTA`) permite registrar miles de transacciones diarias sin degradación de velocidad.
- **Consistencia y Automatización:** Los disparadores y vistas garantizan que la información consultada por la capa de presentación refleje en tiempo real el inventario físico disponible.